

FIREWALLD NOTES



Firewalld is the new concept and default tool to manage the host based Firewall in Centos/RHEL7.0. In earlier version, *iptables* was used to manage the firewall. The *iptables* service still exist, but it should not be used to manage the firewall.

Disabling iptables service

Firewalld is incompatible with iptables and should never be used with it. Running both services simultaneously will mess-up the firewall. Whatever service you choose, disable and mask the other.

There are three iptables services:-

1. iptables for IPv4 configuration
2. ip6tables for IPv6 configuration
3. ebtables for software bridge

We can disable these services individually with following commands

```
systemctl disable iptables
```

```
systemctl disable ip6tables
```

```
systemctl disable ebtables
```

In multi-user environment, other user may enable iptables service again accidentally or intentionally. To stop iptables service completely we can mask it. A masked service cannot be started manually or automatically.

To mask all iptables services we can use following commands

```
systemctl mask iptables
```

```
systemctl mask ip6tables
```

```
systemctl mask ebtables
```

Enabling firewalld Service

If you have just disabled the active iptables service and trying to enable the firewalld service then firewalld service may not be start. Switching from iptables to firewalld requires a system restart. First disable and mask active iptables services and restart the system. Then enable and start the firewalld service.

To check the current status of firewalld service we can use following command.

```
#systemctl status firewalld
```

```
#firewall-cmd --state
```

If firewalld service is masked, it will never start. We have to unmasked the firewalld service before we can start it. To unmask the firewalld service we can use following command:-

```
#systemctl unmask firewalld
```

Once unmasked, we can start it with following command:-

```
#systemctl start firewalld
```

With start option service will start only for this session. To start it permanently we have to use enable option.

```
#systemctl enable firewalld
```

Zones

The firewalld daemon manages groups of rules using entities called “zones”. Zones are basically sets of rules dictating what traffic should be allowed depending on the level of trust you have in the networks your computer is connected to. Network interfaces are assigned a zone to dictate the behavior that the firewall should allow.

For computers that might move between networks frequently (like laptops), this kind of flexibility provides a good method of changing your rules depending on your environment. You may have strict rules in place prohibiting most traffic when operating on a public WiFi network, while allowing more relaxed restrictions when connected to your home network. For a server, these zones are not as immediately important because the network environment rarely, if ever, changes.

Regardless of how dynamic your network environment may be, it is still useful to be familiar with the general idea behind each of the predefined zones for firewalld. In order from least trusted to most trusted, the predefined zones within firewalld are:

drop: The lowest level of trust. All incoming connections are dropped without reply and only outgoing connections are possible.

block: Similar to the above, but instead of simply dropping connections, incoming requests are rejected with an icmp-host-prohibited or icmp6-adm-prohibited message.

public: Represents public, untrusted networks. You don't trust other computers but may allow selected incoming connections on a case-by-case basis.

external: External networks in the event that you are using the firewall as your gateway. It is configured for NAT masquerading so that your internal network remains private but reachable.

internal: The other side of the external zone, used for the internal portion of a gateway. The computers are fairly trustworthy and some additional services are available.

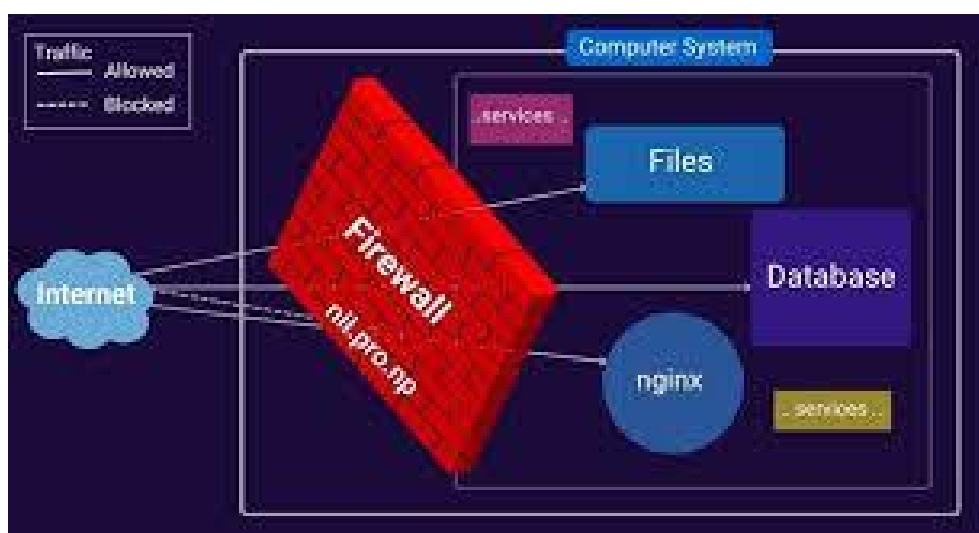
dmz: Used for computers located in a DMZ (isolated computers that will not have access to the rest of your network). Only certain incoming connections are allowed.

work: Used for work machines. Trust most of the computers in the network. A few more services might be allowed.

home: A home environment. It generally implies that you trust most of the other computers and that a few more services will be accepted.

trusted: Trust all of the machines in the network. The most open of the available options and should be used sparingly.

To use the firewall, we can create rules and alter the properties of our zones and then assign our network interfaces to whichever zones are most appropriate.



Getting Familiar with the Current Firewall Rules

Before we begin to make modifications, we should familiarize ourselves with the default environment and rules provided by the daemon.

Exploring the Defaults

We can see which zone is currently selected as the default by typing:

```
firewall-cmd --get-default-zone
```

Output

```
public
```

Since we haven't given firewalld any commands to deviate from the default zone, and none of our interfaces are configured to bind to another zone, that zone will also be the only "active" zone (the zone that is controlling the traffic for our interfaces). We can verify that by typing:

```
firewall-cmd --get-active-zones
```

Output

```
public
```

```
    interfaces: eth0 eth1
```

Here, we can see that our example server has two network interfaces being controlled by the firewall (eth0 and eth1). They are both currently being managed according to the rules defined for the public zone.

How do we know what rules are associated with the public zone though? We can print out the default zone's configuration by typing:

```
# firewall-cmd --list-all
```

Output

```
public (default, active)
  target: default
  icmp-block-inversion: no
  interfaces: eth0 eth1
  sources:
  services: ssh dhcpv6-client
  ports:
  protocols:
  masquerade: no
  forward-ports:
  source-ports:
```

```
icmp-blocks:  
rich rules:
```

We can tell from the output that this zone is both the default and active and that the eth0 and eth1 interfaces are associated with this zone (we already knew all of this from our previous inquiries). However, we can also see that this zone allows for the normal operations associated with a DHCP client (for IP address assignment) and SSH (for remote administration).

Exploring Alternative Zones

Now we have a good idea about the configuration for the default and active zone. We can find out information about other zones as well.

To get a list of the available zones, type:

```
#firewall-cmd --get-zones
```

Output

```
block dmz drop external home internal public trusted work
```

We can see the specific configuration associated with a zone by including the --zone= parameter in our --list-all command:

```
# firewall-cmd --zone=home --list-all
```

Output

home

```
target: default  
icmp-block-inversion: no  
interfaces:  
sources:  
services: dhcpv6-client mdns samba-client ssh  
ports:  
protocols:  
masquerade: no  
forward-ports:  
source-ports:  
icmp-blocks:  
rich rules:
```

You can output all of the zone definitions by using the --list-all-zones option. You will probably want to pipe the output into a pager for easier viewing:

```
#firewall-cmd --list-all-zones | less
```

Selecting Zones for your Interfaces

Unless you have configured your network interfaces otherwise, each interface will be put in the default zone when the firewall is booted.

Changing the Zone of an Interface

You can transition an interface between zones during a session by using the `--zone=` parameter in combination with the `--change-interface=` parameter. As with all commands that modify the firewall, you will need to use `sudo`.

For instance, we can transition our `eth0` interface to the “home” zone by typing this:

```
#firewall-cmd --zone=home --change-interface=eth0
```

Output

Success

Note: Whenever you are transitioning an interface to a new zone, be aware that you are probably modifying the services that will be operational. For instance, here we are moving to the “home” zone, which has SSH available. This means that our connection shouldn’t drop. Some other zones do not have SSH enabled by default and if your connection is dropped while using one of these zones, you could find yourself unable to log back in.

We can verify that this was successful by asking for the active zones again:

```
# firewall-cmd --get-active-zones
```

Output

home

interfaces: eth0

public

interfaces: eth1

Adjusting the Default Zone

If all of your interfaces can best be handled by a single zone, it’s probably easier to select the best default zone and then use that for your configuration.

You can change the default zone with the `--set-default-zone=` parameter. This will immediately change any interface that had fallen back on the default to the new zone:

```
# firewall-cmd --set-default-zone=home
```

Output

```
success
```

Managing Firewalld and Configuring Rules

Now that we have firewalld running, we can get down to set the configuration. We can open ports, allow services, whitelist IPs for access, and more. In all of these examples, we include the `--permanent` flag. This is important to make sure a rule is saved even after you restart firewalld, or reboot the server. Once you're done adding new rules, you need to reload the firewall to make the new rules active.

Add a Port for TCP or UDP

You do have to specify TCP or UDP and to open a port for both. You will need to add rules for each protocol.

```
firewall-cmd --permanent --add-port=22/TCP
```

```
firewall-cmd --permanent --add-port=53/UDP
```

Remove a Port for TCP or UDP

Using a slight variation on the above structure, you can remove a currently open port, effectively closing off that port.

```
firewall-cmd --permanent --remove-port=444/tcp
```

Add a Service

These services assume the default ports configured within the `/etc/services` configuration file; if you wish to use a service on a non-standard port, you will have to open the specific port, as in the example above.

```
firewall-cmd --permanent --add-service=ssh
```

```
firewall-cmd --permanent --add-service=http
```

Remove a Service

As above, you specify the `remove-service` option, and you can close off the port that is defined for that service.

```
firewall-cmd --permanent --remove-service=mysql
```

Whitelist an IP Address

To whitelist or allow access from an IP or range of IPs, you can tell the firewall to add a trusted source.

```
firewall-cmd --permanent --add-source=192.168.1.100
```

You can also allow a range of IPs using what is called CIDR notation. CIDR is outside the scope of this article but is a shorthand that can be used for noting ranges of IP addresses.

```
firewall-cmd --permanent --add-source=192.168.1.0/24
```

Remove a Whitelisted IP Address

To remove a whitelisted IP or IP range, you can use the `--remove-source` option.

```
firewall-cmd --permanent --remove-source=192.168.1.100
```

Block an IP Address

As the `firewall-cmd` tool is mostly used for opening or allowing access, rich rules are needed to block an IP. Rich rules are similar in form to the way `iptables` rules are written.

```
firewall-cmd --permanent --add-rich-rule="rule family='ipv4' source address='192.168.1.100' reject"
```

You can again use CIDR notation also block a range of IP addresses.

```
firewall-cmd --permanent --add-rich-rule="rule family='ipv4' source address='192.168.1.0/24' reject"
```

Whitelist an IP Address for a Specific Port (More Rich Rules)

We have to reach back to `iptables` and create another rich rule; however, we are using the `accept` statement at the end to allow the IP access, rather than reject its access.

```
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="192.168.1.100" port protocol="tcp" port="3306" accept'
```


Removing a Rich Rule

To remove a rich rule, use the option `--remove-rich-rule`, but you have to fully specify which rule is being removed, so it is best to copy and paste the full rule, rather than try to type it all out from memory.

```
firewall-cmd --permanent --remove-rich-rule='rule family="ipv4"
source address="192.168.1.100" port protocol="tcp" port="3306"
accept'
```

Saving Firewall Rules

After you have completed all the additions and subtraction of rules, you need to reload the firewall rules to make them active. To do this, you again use the `firewall-cmd` tool but using the option `--reload`.

```
firewall-cmd --reload
```

Viewing Firewall Rules

After reloading the rules, you can confirm if the new rules are in place correctly with the following.

```
firewall-cmd --list-all
```

Here is an example output from the `--list-all` option, you can see that this server has a number of ports, and services open in the firewall along with a rich rule (that forwards one port to another).

```
[root@centos-7 ~]# firewall-cmd --list-all
public (default, active)
interfaces: enp1s0
sources: 192.168.1.0/24
services: dhcpv6-client dns http https mysql nfs samba smtp ssh
ports: 443/tcp 80/tcp 5900-5902/tcp 83/tcp 444/tcp 3260/tcp
masquerade: no
forward-ports:
icmp-blocks:
rich rules:
rule family="ipv4" source address="192.168.1.0/24" forward-port
port="5423" protocol="tcp" to-port="80"
```

More Options

Find configuration fire

```
#rpm -qc firewalld
```

```
/etc/firewalld/firewalld.conf
```

Service Directoy

```
/usr/lib/firewalld/services/
```

Allow multiport port at once

```
firewall-cmd --add port={3306/tcp,6000/tcp}
```

```
firewall-cmd --add-port=5000-5010/tcp
```

```
firewall-cmd --add-service=(http,https,ldap)
```

Port forwarding:

Port 8080 to 80

```
firewall-cmd --add-forward-port=8080:proto=tcp:toport=80
```

```
firewall-cmd --list-all
```

Rich Rules

```
firewall-cmd --add-rich-rule='rule family=i"pv4" source  
address='192.168.0.3' accept'
```

```
firewall-cmd --add-rich-rule='rule family=i"pv4" source  
address='192.168.0.4' drop'
```

```
firewall-cmd --list-all
```

Check interfaces/zone status

```
firewall-cmd --get-zone-of-interfaces
```

```
firewall-cmd --zone=public --list-interfaces
```

Panic mode

```
firewall-cmd --query-panic
```

```
firewall-cmd --panic-on
```

```
firewall-cmd --panic-off
```